

EXPERIMENT 4: ALPHA-BETA PRUNING ALGORITHM

Aim

To write a Python program to implement the **Alpha-Beta pruning algorithm for gaming**.

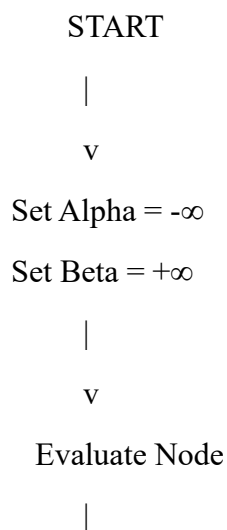
Objective

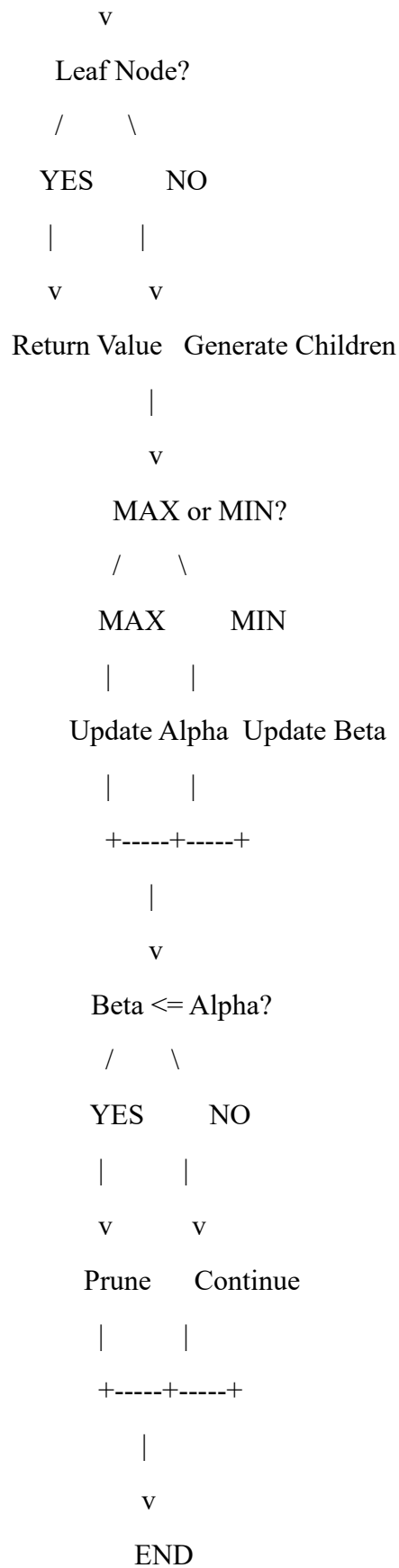
- To understand optimization of the Minimax algorithm.
- To define alpha and beta values.
- To eliminate unnecessary branches.
- To reduce the number of nodes evaluated.
- To determine the optimal game value efficiently.

Algorithm

1. Start with the root node.
2. Initialize $\alpha = -\infty$ and $\beta = +\infty$.
3. Generate the child nodes.
4. For MAX nodes, select the maximum value.
5. Update α with the maximum value.
6. For MIN nodes, select the minimum value.
7. Update β with the minimum value.
8. If $\beta \leq \alpha$, stop evaluating the remaining children.
9. This process is called pruning.
10. Return the optimal value.

Flowchart





Python Program

Writing

Alpha-Beta Pruning

```
def alphabeta(depth, node, maximizing, values, alpha, beta):
```

```
    # Leaf node
```

```
    if depth == 3:
```

```
        return values[node]
```

```
    if maximizing:
```

```
        best = -1000
```

```
        for i in range(2):
```

```
            value = alphabeta(
```

```
                depth + 1,
```

```
                node * 2 + i,
```

```
                False,
```

```
                values,
```

```
                alpha,
```

```
                beta
```

```
            )
```

```
            best = max(best, value)
```

```
            alpha = max(alpha, best)
```

```
        # Alpha-Beta pruning
```

```
        if beta <= alpha:
```

```
            break
```

```
    return best
```

else:

best = 1000

for i in range(2):

value = alphabeta(

depth + 1,

node * 2 + i,

True,

values,

alpha,

beta

)

best = min(best, value)

beta = min(beta, best)

Alpha-Beta pruning

if beta <= alpha:

break

return best

values = [3, 5, 6, 9, 1, 2, 0, -1]

result = alphabeta(

0, 0, True,

values,

-1000,

1000

)

```
print("Leaf node values:", values)
```

```
print("Optimal value:", result)
```

Sample Output

Leaf node values: [3, 5, 6, 9, 1, 2, 0, -1]

Optimal value: 5

Result

Thus, the **Alpha-Beta pruning algorithm** was successfully implemented and the optimal value was obtained while eliminating unnecessary branches.